

200

Java Coding Problems with Full Solutions

Master Java Programming & Crack Coding Interviews 🚀🔥

Beginner to Advanced Problems

Clean & Optimized Solutions

Logic Building & Problem Solving

Interview-Focused Preparation

Perfect for Students & Java Developers 🖥️🔥

1. Reverse a String

```
public class Main {  
    public static void main(String[] args) {  
        String str = "hello";  
        String rev = "";  
  
        for (int i = str.length() - 1; i >= 0; i--) {  
            rev += str.charAt(i);  
        }  
  
        System.out.println(rev);  
    }  
}
```

2. Check Palindrome

```
public class Main {  
    public static void main(String[] args) {  
        String str = "madam";  
        String rev = new StringBuilder(str).reverse().toString();  
  
        if (str.equals(rev))  
            System.out.println("Palindrome");  
        else  
            System.out.println("Not Palindrome");  
    }  
}
```

3. Factorial of a Number

```
public class Main {  
    public static void main(String[] args) {  
        int n = 5;  
        int fact = 1;  
  
        for (int i = 1; i <= n; i++) {  
            fact *= i;  
        }  
  
        System.out.println(fact);  
    }  
}
```

4. Fibonacci Series

```
public class Main {  
    public static void main(String[] args) {  
        int n = 10, a = 0, b = 1;  
  
        for (int i = 0; i < n; i++) {  
            System.out.print(a + " ");  
            int c = a + b;  
            a = b;  
            b = c;  
        }  
    }  
}
```

5. Prime Number Check

```
public class Main {  
    public static void main(String[] args) {  
        int n = 29;  
        boolean isPrime = true;  
  
        for (int i = 2; i <= n / 2; i++) {  
            if (n % i == 0) {  
                isPrime = false;  
                break;  
            }  
        }  
  
        System.out.println(isPrime ? "Prime" : "Not Prime");  
    }  
}
```

6. Swap Two Numbers

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5, b = 10;  
  
        a = a + b;  
        b = a - b;  
        a = a - b;  
  
        System.out.println(a + " " + b);  
    }  
}
```

7. Find Largest Element in Array

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 5, 3, 9, 2};  
        int max = arr[0];  
  
        for (int num : arr) {  
            if (num > max) max = num;  
        }  
  
        System.out.println(max);  
    }  
}
```

8. Sum of Array Elements

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4};  
        int sum = 0;  
  
        for (int num : arr) {  
            sum += num;  
        }  
  
        System.out.println(sum);  
    }  
}
```

9. Count Vowels in String

```
public class Main {  
    public static void main(String[] args) {  
        String str = "hello world";  
        int count = 0;  
  
        for (char c : str.toCharArray()) {  
            if ("aeiou".indexOf(c) != -1)  
                count++;  
        }  
  
        System.out.println(count);  
    }  
}
```

10. Check Armstrong Number

```
public class Main {
    public static void main(String[] args) {
        int n = 153, sum = 0, temp = n;

        while (temp != 0) {
            int digit = temp % 10;
            sum += digit * digit * digit;
            temp /= 10;
        }

        System.out.println(sum == n ? "Armstrong" : "Not Armstrong");
    }
}
```

11. Count Digits in Number

```
public class Main {
    public static void main(String[] args) {
        int n = 12345, count = 0;

        while (n != 0) {
            count++;
            n /= 10;
        }

        System.out.println(count);
    }
}
```

12. Reverse Number

```
public class Main {
    public static void main(String[] args) {
        int n = 123, rev = 0;

        while (n != 0) {
            rev = rev * 10 + n % 10;
            n /= 10;
        }

        System.out.println(rev);
    }
}
```

13. Check Even or Odd

```
public class Main {  
    public static void main(String[] args) {  
        int n = 7;  
        System.out.println(n % 2 == 0 ? "Even" : "Odd");  
    }  
}
```

14. Sum of Digits

```
public class Main {  
    public static void main(String[] args) {  
        int n = 123, sum = 0;  
  
        while (n != 0) {  
            sum += n % 10;  
            n /= 10;  
        }  
  
        System.out.println(sum);  
    }  
}
```

15. GCD of Two Numbers

```
public class Main {  
    public static void main(String[] args) {  
        int a = 12, b = 18;  
  
        while (b != 0) {  
            int temp = b;  
            b = a % b;  
            a = temp;  
        }  
  
        System.out.println(a);  
    }  
}
```

16. LCM of Two Numbers

```
public class Main {  
    public static void main(String[] args) {  
        int a = 12, b = 18;  
        int lcm = (a * b) / gcd(a, b);  
        System.out.println(lcm);  
    }  
  
    static int gcd(int a, int b) {  
        while (b != 0) {  
            int temp = b;  
            b = a % b;  
            a = temp;  
        }  
        return a;  
    }  
}
```

17. Binary to Decimal

```
public class Main {  
    public static void main(String[] args) {  
        String binary = "1010";  
        int decimal = Integer.parseInt(binary, 2);  
        System.out.println(decimal);  
    }  
}
```

18. Decimal to Binary

```
public class Main {  
    public static void main(String[] args) {  
        int num = 10;  
        System.out.println(Integer.toBinaryString(num));  
    }  
}
```

19. Linear Search

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 3, 5, 7};
        int key = 5;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                System.out.println("Found at " + i);
                return;
            }
        }

        System.out.println("Not Found");
    }
}
```

20. Bubble Sort

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 1};

        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr.length - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }

        System.out.println(Arrays.toString(arr));
    }
}
```


21. Remove Duplicates from Array

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 2, 3, 4};
        Set<Integer> set = new LinkedHashSet<>();

        for (int num : arr) set.add(num);

        System.out.println(set);
    }
}
```

22. Count Words in String

```
public class Main {
    public static void main(String[] args) {
        String str = "Hello Java World";
        String[] words = str.split(" ");
        System.out.println(words.length);
    }
}
```

23. Find Second Largest Number

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 4, 45};
        int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;

        for (int num : arr) {
            if (num > first) {
                second = first;
                first = num;
            } else if (num > second && num != first) {
                second = num;
            }
        }

        System.out.println(second);
    }
}
```

24. Check Anagram

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        String s1 = "listen", s2 = "silent";

        char[] a = s1.toCharArray();
        char[] b = s2.toCharArray();

        Arrays.sort(a);
        Arrays.sort(b);

        System.out.println(Arrays.equals(a, b));
    }
}
```

25. Matrix Addition

```
public class Main {
    public static void main(String[] args) {
        int[][] a = {{1,2},{3,4}};
        int[][] b = {{5,6},{7,8}};
        int[][] c = new int[2][2];

        for(int i=0;i<2;i++){
            for(int j=0;j<2;j++){
                c[i][j]=a[i][j]+b[i][j];
                System.out.print(c[i][j]+" ");
            }
            System.out.println();
        }
    }
}
```

26. Check Leap Year

```
public class Main {
    public static void main(String[] args) {
        int year = 2024;

        if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
            System.out.println("Leap Year");
        else
            System.out.println("Not Leap Year");
    }
}
```

27. Print Multiplication Table

```
public class Main {  
    public static void main(String[] args) {  
        int n = 5;  
  
        for (int i = 1; i <= 10; i++) {  
            System.out.println(n + " x " + i + " = " + (n * i));  
        }  
    }  
}
```

28. Find Factor of Number

```
public class Main {  
    public static void main(String[] args) {  
        int n = 12;  
  
        for (int i = 1; i <= n; i++) {  
            if (n % i == 0)  
                System.out.print(i + " ");  
        }  
    }  
}
```

29. Sum of First N Natural Numbers

```
public class Main {  
    public static void main(String[] args) {  
        int n = 10, sum = 0;  
  
        for (int i = 1; i <= n; i++) {  
            sum += i;  
        }  
  
        System.out.println(sum);  
    }  
}
```

30. Check Perfect Number

```
public class Main {
    public static void main(String[] args) {
        int n = 28, sum = 0;

        for (int i = 1; i < n; i++) {
            if (n % i == 0)
                sum += i;
        }

        System.out.println(sum == n ? "Perfect Number" : "Not Perfect");
    }
}
```

31. Count Frequency of Elements

```
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 2, 3, 3, 3};

        HashMap<Integer, Integer> map = new HashMap<>();

        for (int num : arr) {
            map.put(num, map.getDefault(num, 0) + 1);
        }

        System.out.println(map);
    }
}
```

32. Find Missing Number in Array

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 4, 5};
        int n = 5;
        int sum = n * (n + 1) / 2;

        int arrSum = 0;
        for (int num : arr) arrSum += num;

        System.out.println(sum - arrSum);
    }
}
```

33. Check Sorted Array

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4};  
        boolean sorted = true;  
  
        for (int i = 1; i < arr.length; i++) {  
            if (arr[i] < arr[i - 1]) {  
                sorted = false;  
                break;  
            }  
        }  
  
        System.out.println(sorted);  
    }  
}
```

34. Rotate Array by One

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4};  
  
        int last = arr[arr.length - 1];  
  
        for (int i = arr.length - 1; i > 0; i--) {  
            arr[i] = arr[i - 1];  
        }  
  
        arr[0] = last;  
  
        for (int num : arr)  
            System.out.print(num + " ");  
    }  
}
```

35. Find Duplicate Element

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 2};
        HashSet<Integer> set = new HashSet<>();

        for (int num : arr) {
            if (!set.add(num)) {
                System.out.println("Duplicate: " + num);
                break;
            }
        }
    }
}
```

36. Reverse Array

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3};

        for (int i = arr.length - 1; i >= 0; i--) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

37. Merge Two Arrays

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] a = {1, 2};
        int[] b = {3, 4};

        int[] c = new int[a.length + b.length];

        System.arraycopy(a, 0, c, 0, a.length);
        System.arraycopy(b, 0, c, a.length, b.length);

        System.out.println(Arrays.toString(c));
    }
}
```

38. Find Common Elements

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[] b = {2, 3, 4};

        HashSet<Integer> set = new HashSet<>();

        for (int num : a) set.add(num);

        for (int num : b) {
            if (set.contains(num))
                System.out.print(num + " ");
        }
    }
}
```

39. Remove Spaces from String

```
public class Main {
    public static void main(String[] args) {
        String str = "Hello World";
        str = str.replace(" ", "");
        System.out.println(str);
    }
}
```

40. Convert String to Uppercase

```
public class Main {
    public static void main(String[] args) {
        String str = "java";
        System.out.println(str.toUpperCase());
    }
}
```

41. Count Characters in String

```
public class Main {
    public static void main(String[] args) {
        String str = "hello";
        System.out.println(str.length());
    }
}
```

42. Find First Non-Repeating Character

```
import java.util.LinkedHashMap;

public class Main {
    public static void main(String[] args) {
        String str = "swiss";
        LinkedHashMap<Character, Integer> map = new LinkedHashMap<>();

        for (char c : str.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }

        for (char c : map.keySet()) {
            if (map.get(c) == 1) {
                System.out.println(c);
                break;
            }
        }
    }
}
```

43. Check String Contains Only Digits

```
public class Main {
    public static void main(String[] args) {
        String str = "12345";
        boolean isDigit = str.matches("\\d+");
        System.out.println(isDigit);
    }
}
```

44. Convert String to Integer

```
public class Main {
    public static void main(String[] args) {
        String str = "100";
        int num = Integer.parseInt(str);
        System.out.println(num);
    }
}
```


45. Find Largest Word in Sentence

```
public class Main {  
    public static void main(String[] args) {  
        String str = "Java is powerful language";  
        String[] words = str.split(" ");  
  
        String largest = "";  
        for (String word : words) {  
            if (word.length() > largest.length())  
                largest = word;  
        }  
  
        System.out.println(largest);  
    }  
}
```

46. Check Two Strings Equal

```
public class Main {  
    public static void main(String[] args) {  
        String a = "java", b = "java";  
        System.out.println(a.equals(b));  
    }  
}
```

47. Toggle Case of Characters

```
public class Main {  
    public static void main(String[] args) {  
        String str = "Java";  
  
        StringBuilder result = new StringBuilder();  
  
        for (char c : str.toCharArray()) {  
            if (Character.isUpperCase(c))  
                result.append(Character.toLowerCase(c));  
            else  
                result.append(Character.toUpperCase(c));  
        }  
  
        System.out.println(result);  
    }  
}
```

48. Count Occurrence of Character

```
public class Main {  
    public static void main(String[] args) {  
        String str = "banana";  
        char ch = 'a';  
        int count = 0;  
  
        for (char c : str.toCharArray()) {  
            if (c == ch) count++;  
        }  
  
        System.out.println(count);  
    }  
}
```

49. Remove Character from String

```
public class Main {  
    public static void main(String[] args) {  
        String str = "banana";  
        str = str.replace("a", "");  
        System.out.println(str);  
    }  
}
```

50. Sort Characters in String

```
import java.util.Arrays;  
  
public class Main {  
    public static void main(String[] args) {  
        String str = "dcba";  
  
        char[] arr = str.toCharArray();  
        Arrays.sort(arr);  
  
        System.out.println(new String(arr));  
    }  
}
```

51. Sum of Even Numbers in Array

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4, 6};  
        int sum = 0;  
  
        for (int num : arr) {  
            if (num % 2 == 0) sum += num;  
        }  
  
        System.out.println(sum);  
    }  
}
```

52. Sum of Odd Numbers in Array

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4, 5};  
        int sum = 0;  
  
        for (int num : arr) {  
            if (num % 2 != 0) sum += num;  
        }  
  
        System.out.println(sum);  
    }  
}
```

53. Find Smallest Element

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {5, 2, 8, 1};  
        int min = arr[0];  
  
        for (int num : arr) {  
            if (num < min) min = num;  
        }  
  
        System.out.println(min);  
    }  
}
```

54. Copy Array

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        int[] copy = Arrays.copyOf(arr, arr.length);

        System.out.println(Arrays.toString(copy));
    }
}
```

55. Check Array Equality

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[] b = {1, 2, 3};

        System.out.println(Arrays.equals(a, b));
    }
}
```

56. Find Index of Element

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30};
        int key = 20;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                System.out.println(i);
                break;
            }
        }
    }
}
```

57. Count Positive and Negative Numbers

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {-1, 2, -3, 4};  
        int pos = 0, neg = 0;  
  
        for (int num : arr) {  
            if (num >= 0) pos++;  
            else neg++;  
        }  
  
        System.out.println("Positive: " + pos + ", Negative: " + neg);  
    }  
}
```

58. Move Zeros to End

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {0, 1, 0, 3, 12};  
        int index = 0;  
  
        for (int num : arr) {  
            if (num != 0) arr[index++] = num;  
        }  
  
        while (index < arr.length) {  
            arr[index++] = 0;  
        }  
  
        for (int num : arr)  
            System.out.print(num + " ");  
    }  
}
```

59. Find Intersection of Arrays

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[] b = {2, 3, 4};

        HashSet<Integer> set = new HashSet<>();
        for (int num : a) set.add(num);

        for (int num : b) {
            if (set.contains(num))
                System.out.print(num + " ");
        }
    }
}
```

60. Find Union of Arrays

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        int[] a = {1, 2};
        int[] b = {2, 3};

        HashSet<Integer> set = new HashSet<>();

        for (int num : a) set.add(num);
        for (int num : b) set.add(num);

        System.out.println(set);
    }
}
```

61. Reverse String Using Recursion

```
public class Main {
    static void reverse(String str) {
        if (str.isEmpty()) return;
        reverse(str.substring(1));
        System.out.print(str.charAt(0));
    }

    public static void main(String[] args) {
        reverse("hello");
    }
}
```

62. Factorial Using Recursion

```
public class Main {  
    static int fact(int n) {  
        if (n == 0) return 1;  
        return n * fact(n - 1);  
    }  
  
    public static void main(String[] args) {  
        System.out.println(fact(5));  
    }  
}
```

63. Fibonacci Using Recursion

```
public class Main {  
    static int fib(int n) {  
        if (n <= 1) return n;  
        return fib(n - 1) + fib(n - 2);  
    }  
  
    public static void main(String[] args) {  
        System.out.println(fib(6));  
    }  
}
```

64. Power of Number (Recursion)

```
public class Main {  
    static int power(int a, int b) {  
        if (b == 0) return 1;  
        return a * power(a, b - 1);  
    }  
  
    public static void main(String[] args) {  
        System.out.println(power(2, 3));  
    }  
}
```

65. Sum of Digits (Recursion)

```
public class Main {  
    static int sum(int n) {  
        if (n == 0) return 0;  
        return n % 10 + sum(n / 10);  
    }  
  
    public static void main(String[] args) {  
        System.out.println(sum(123));  
    }  
}
```

66. Binary Search

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int key = 4;
        int low = 0, high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (arr[mid] == key) {
                System.out.println(mid);
                return;
            } else if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }
}
```

67. Selection Sort

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] arr = {4, 2, 7, 1};

        for (int i = 0; i < arr.length; i++) {
            int min = i;

            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[min])
                    min = j;
            }

            int temp = arr[i];
            arr[i] = arr[min];
            arr[min] = temp;
        }

        System.out.println(Arrays.toString(arr));
    }
}
```


68. Insertion Sort

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 3, 1, 4};

        for (int i = 1; i < arr.length; i++) {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = key;
        }

        System.out.println(Arrays.toString(arr));
    }
}
```

69. Check Prime (Optimized)

```
public class Main {
    public static void main(String[] args) {
        int n = 29;
        boolean prime = true;

        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                prime = false;
                break;
            }
        }

        System.out.println(prime);
    }
}
```

70. Find All Prime Numbers up to N

```
public class Main {
    public static void main(String[] args) {
        int n = 20;

        for (int i = 2; i <= n; i++) {
            boolean prime = true;

            for (int j = 2; j <= Math.sqrt(i); j++) {
                if (i % j == 0) {
                    prime = false;
                    break;
                }
            }

            if (prime) System.out.print(i + " ");
        }
    }
}
```

71. Transpose Matrix

```
public class Main {
    public static void main(String[] args) {
        int[][] mat = {{1,2,3},{4,5,6}};

        for(int i=0;i<3;i++){
            for(int j=0;j<2;j++){
                System.out.print(mat[j][i]+" ");
            }
            System.out.println();
        }
    }
}
```

72. Diagonal Sum of Matrix

```
public class Main {
    public static void main(String[] args) {
        int[][] mat = {{1,2,3},{4,5,6},{7,8,9}};
        int sum = 0;

        for(int i=0;i<mat.length;i++){
            sum += mat[i][i];
        }

        System.out.println(sum);
    }
}
```

73. Check Identity Matrix

```
public class Main {
    public static void main(String[] args) {
        int[][] mat = {{1,0,0},{0,1,0},{0,0,1}};
        boolean identity = true;

        for(int i=0;i<mat.length;i++){
            for(int j=0;j<mat.length;j++){
                if((i==j && mat[i][j]!=1) || (i!=j && mat[i][j]!=0)){
                    identity = false;
                }
            }
        }

        System.out.println(identity);
    }
}
```

74. Check Sparse Matrix

```
public class Main {
    public static void main(String[] args) {
        int[][] mat = {{0,0,3},{0,0,0},{1,0,0}};
        int zero = 0;

        for(int[] row : mat){
            for(int val : row){
                if(val == 0) zero++;
            }
        }

        if(zero > (mat.length * mat[0].length) / 2)
            System.out.println("Sparse Matrix");
    }
}
```

75. Matrix Multiplication

```
public class Main {
    public static void main(String[] args) {
        int[][] a = {{1,2},{3,4}};
        int[][] b = {{5,6},{7,8}};
        int[][] c = new int[2][2];

        for(int i=0;i<2;i++){
            for(int j=0;j<2;j++){
                for(int k=0;k<2;k++){
                    c[i][j] += a[i][k]*b[k][j];
                }
                System.out.print(c[i][j]+" ");
            }
            System.out.println();
        }
    }
}
```

76. Create a Class and Object

```
class Student {
    String name = "John";
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.name);
    }
}
```

77. Constructor Example

```
class Student {
    String name;

    Student(String n) {
        name = n;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Alice");
        System.out.println(s.name);
    }
}
```

78. Method Overloading

```
class MathOp {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MathOp obj = new MathOp();  
        System.out.println(obj.add(2, 3));  
        System.out.println(obj.add(2, 3, 4));  
    }  
}
```

79. Method Overriding

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Bark");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        a.sound();  
    }  
}
```

80. Single Inheritance

```
class A {  
    void show() {  
        System.out.println("Class A");  
    }  
}  
  
class B extends A {  
    void display() {  
        System.out.println("Class B");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        B obj = new B();  
        obj.show();  
        obj.display();  
    }  
}
```

81. Multilevel Inheritance

```
class A {  
    void a() { System.out.println("A"); }  
}  
  
class B extends A {  
    void b() { System.out.println("B"); }  
}  
  
class C extends B {  
    void c() { System.out.println("C"); }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.a();  
        obj.b();  
        obj.c();  
    }  
}
```

82. Hierarchical Inheritance

```
class A {  
    void show() {  
        System.out.println("A");  
    }  
}  
  
class B extends A {}  
class C extends A {}  
  
public class Main {  
    public static void main(String[] args) {  
        B b = new B();  
        C c = new C();  
        b.show();  
        c.show();  
    }  
}
```

83. Interface Example

```
interface Animal {  
    void sound();  
}  
  
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Bark");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        a.sound();  
    }  
}
```

84. Abstract Class

```
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s = new Circle();
        s.draw();
    }
}
```

85. Encapsulation

```
class Person {
    private String name;

    public void setName(String n) {
        name = n;
    }

    public String getName() {
        return name;
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.setName("John");
        System.out.println(p.getName());
    }
}
```


86. Static Keyword

```
class Test {
    static int count = 0;

    Test() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) {
        new Test();
        new Test();
        System.out.println(Test.count);
    }
}
```

87. Final Keyword

```
class Test {
    final int x = 10;
}

public class Main {
    public static void main(String[] args) {
        Test t = new Test();
        System.out.println(t.x);
    }
}
```

88. This Keyword

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Ram");
        System.out.println(s.name);
    }
}
```

89. Super Keyword

```
class A {
    int x = 10;
}

class B extends A {
    int x = 20;

    void show() {
        System.out.println(super.x);
    }
}

public class Main {
    public static void main(String[] args) {
        B obj = new B();
        obj.show();
    }
}
```

90. Exception Handling

```
public class Main {
    public static void main(String[] args) {
        try {
            int x = 10 / 0;
        } catch (Exception e) {
            System.out.println("Error occurred");
        }
    }
}
```

91. Custom Exception

```
class MyException extends Exception {
    MyException(String msg) {
        super(msg);
    }
}

public class Main {
    public static void main(String[] args) {
        try {
            throw new MyException("Custom Error");
        } catch (MyException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

92. File Writing

```
import java.io.FileWriter;

public class Main {
    public static void main(String[] args) throws Exception {
        FileWriter fw = new FileWriter("test.txt");
        fw.write("Hello");
        fw.close();
    }
}
```

93. File Reading

```
import java.io.File;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) throws Exception {
        File file = new File("test.txt");
        Scanner sc = new Scanner(file);

        while (sc.hasNextLine()) {
            System.out.println(sc.nextLine());
        }

        sc.close();
    }
}
```

94. ArrayList Example

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);

        System.out.println(list);
    }
}
```

95. LinkedList Example

```
import java.util.LinkedList;

public class Main {
    public static void main(String[] args) {
        LinkedList<Integer> list = new LinkedList<>();
        list.add(1);
        list.add(2);

        System.out.println(list);
    }
}
```

96. HashSet Example

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        HashSet<Integer> set = new HashSet<>();
        set.add(1);
        set.add(1);

        System.out.println(set);
    }
}
```

97. HashMap Example

```
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(1, "A");

        System.out.println(map);
    }
}
```

98. Stack Example

```
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(1);
        stack.push(2);

        System.out.println(stack.pop());
    }
}
```

99. Queue Example

```
import java.util.LinkedList;
import java.util.Queue;

public class Main {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();
        q.add(1);
        q.add(2);

        System.out.println(q.poll());
    }
}
```

100. Lambda Expression

```
interface A {
    void show();
}

public class Main {
    public static void main(String[] args) {
        A obj = () -> System.out.println("Lambda");
        obj.show();
    }
}
```

101. Two Sum Problem

```
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        int[] arr = {2, 7, 11, 15};
        int target = 9;

        HashMap<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < arr.length; i++) {
            int diff = target - arr[i];
            if (map.containsKey(diff)) {
                System.out.println(map.get(diff) + " " + i);
                return;
            }
            map.put(arr[i], i);
        }
    }
}
```

102. Maximum Subarray Sum (Kadane's Algorithm)

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {-2,1,-3,4,-1,2,1,-5,4};
        int maxSum = arr[0], current = arr[0];

        for (int i = 1; i < arr.length; i++) {
            current = Math.max(arr[i], current + arr[i]);
            maxSum = Math.max(maxSum, current);
        }

        System.out.println(maxSum);
    }
}
```

103. Best Time to Buy and Sell Stock

```
public class Main {  
    public static void main(String[] args) {  
        int[] prices = {7,1,5,3,6,4};  
        int min = prices[0], profit = 0;  
  
        for (int price : prices) {  
            if (price < min) min = price;  
            else profit = Math.max(profit, price - min);  
        }  
  
        System.out.println(profit);  
    }  
}
```

104. Move Negative Numbers to One Side

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, -2, 3, -4, 5};  
        int j = 0;  
  
        for (int i = 0; i < arr.length; i++) {  
            if (arr[i] < 0) {  
                int temp = arr[i];  
                arr[i] = arr[j];  
                arr[j] = temp;  
                j++;  
            }  
        }  
  
        for (int num : arr)  
            System.out.print(num + " ");  
    }  
}
```

105. Longest Substring Without Repeating Characters

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        String s = "abcabcbb";
        HashSet<Character> set = new HashSet<>();

        int left = 0, max = 0;

        for (int right = 0; right < s.length(); right++) {
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left++));
            }
            set.add(s.charAt(right));
            max = Math.max(max, right - left + 1);
        }

        System.out.println(max);
    }
}
```

106. Subarray with Given Sum (Positive Integers)

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1,2,3,7,5};
        int target = 12;

        int start = 0, sum = 0;

        for (int end = 0; end < arr.length; end++) {
            sum += arr[end];

            while (sum > target) {
                sum -= arr[start++];
            }

            if (sum == target) {
                System.out.println(start + " " + end);
                return;
            }
        }
    }
}
```


107. Count Subarrays with Given Sum

```
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        int[] arr = {1,1,1};
        int k = 2, count = 0, sum = 0;

        HashMap<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);

        for (int num : arr) {
            sum += num;

            if (map.containsKey(sum - k))
                count += map.get(sum - k);

            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }

        System.out.println(count);
    }
}
```

108. Majority Element (> n/2)

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {3,3,4};
        int count = 0, candidate = 0;

        for (int num : arr) {
            if (count == 0) candidate = num;
            count += (num == candidate) ? 1 : -1;
        }

        System.out.println(candidate);
    }
}
```

109. Missing Number (0 to n)

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {3,0,1};  
        int n = arr.length;  
        int sum = n * (n + 1) / 2;  
  
        int arrSum = 0;  
        for (int num : arr) arrSum += num;  
  
        System.out.println(sum - arrSum);  
    }  
}
```

110. Find Duplicate Number

```
import java.util.HashSet;  
  
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1,3,4,2,2};  
        HashSet<Integer> set = new HashSet<>();  
  
        for (int num : arr) {  
            if (!set.add(num)) {  
                System.out.println(num);  
                break;  
            }  
        }  
    }  
}
```

111. Product of Array Except Self

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1,2,3,4};
        int n = arr.length;

        int[] res = new int[n];
        res[0] = 1;

        for (int i = 1; i < n; i++)
            res[i] = res[i - 1] * arr[i - 1];

        int right = 1;

        for (int i = n - 1; i >= 0; i--) {
            res[i] *= right;
            right *= arr[i];
        }

        for (int num : res)
            System.out.print(num + " ");
    }
}
```

112. Maximum Product Subarray

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {2,3,-2,4};
        int max = arr[0], min = arr[0], result = arr[0];

        for (int i = 1; i < arr.length; i++) {
            int temp = max;
            max = Math.max(arr[i], Math.max(max * arr[i], min * arr[i]));
            min = Math.min(arr[i], Math.min(temp * arr[i], min * arr[i]));
            result = Math.max(result, max);
        }

        System.out.println(result);
    }
}
```

113. Rotate Array by K Steps

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1,2,3,4,5};
        int k = 2;
        int n = arr.length;

        k = k % n;

        reverse(arr, 0, n - 1);
        reverse(arr, 0, k - 1);
        reverse(arr, k, n - 1);

        for (int num : arr)
            System.out.print(num + " ");
    }

    static void reverse(int[] arr, int i, int j) {
        while (i < j) {
            int temp = arr[i];
            arr[i++] = arr[j];
            arr[j--] = temp;
        }
    }
}
```

114. Trapping Rain Water

```
public class Main {
    public static void main(String[] args) {
        int[] height = {0,1,0,2,1,0,1,3};
        int left = 0, right = height.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;

        while (left < right) {
            if (height[left] < height[right]) {
                leftMax = Math.max(leftMax, height[left]);
                water += leftMax - height[left];
                left++;
            } else {
                rightMax = Math.max(rightMax, height[right]);
                water += rightMax - height[right];
                right--;
            }
        }

        System.out.println(water);
    }
}
```

115. Container With Most Water

```
public class Main {
    public static void main(String[] args) {
        int[] h = {1,8,6,2,5,4,8,3,7};
        int left = 0, right = h.length - 1;
        int max = 0;

        while (left < right) {
            int area = Math.min(h[left], h[right]) * (right - left);
            max = Math.max(max, area);

            if (h[left] < h[right]) left++;
            else right--;
        }

        System.out.println(max);
    }
}
```

116. Merge Intervals

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[][] intervals = {{1,3},{2,6},{8,10}};
        Arrays.sort(intervals, (a,b) -> a[0] - b[0]);

        List<int[]> result = new ArrayList<>();

        for (int[] interval : intervals) {
            if (result.isEmpty() || result.get(result.size()-1)[1] < interval[0]) {
                result.add(interval);
            } else {
                result.get(result.size()-1)[1] =
                    Math.max(result.get(result.size()-1)[1], interval[1]);
            }
        }

        for (int[] i : result)
            System.out.println(i[0] + " " + i[1]);
    }
}
```

117. Set Matrix Zeroes

```
public class Main {
    public static void main(String[] args) {
        int[][] mat = {{1,1,1},{1,0,1},{1,1,1}};
        boolean row0 = false, col0 = false;

        for(int i=0;i<mat.length;i++)
            if(mat[i][0]==0) col0=true;

        for(int j=0;j<mat[0].length;j++)
            if(mat[0][j]==0) row0=true;

        for(int i=1;i<mat.length;i++){
            for(int j=1;j<mat[0].length;j++){
                if(mat[i][j]==0){
                    mat[i][0]=mat[0][j]=0;
                }
            }
        }

        for(int i=1;i<mat.length;i++){
            for(int j=1;j<mat[0].length;j++){
                if(mat[i][0]==0 || mat[0][j]==0)
                    mat[i][j]=0;
            }
        }
    }
}
```

118. Spiral Matrix

```
public class Main {  
    public static void main(String[] args) {  
        int[][] mat = {{1,2,3},{4,5,6},{7,8,9}};  
        int top=0, bottom=2, left=0, right=2;  
  
        while(top<=bottom && left<=right){  
            for(int i=left;i<=right;i++){  
                System.out.print(mat[top][i]+" ");  
                top++;  
  
                for(int i=top;i<=bottom;i++){  
                    System.out.print(mat[i][right]+" ");  
                    right--;  
  
                    for(int i=right;i>=left;i--){  
                        System.out.print(mat[bottom][i]+" ");  
                        bottom--;  
  
                        for(int i=bottom;i>=top;i--){  
                            System.out.print(mat[i][left]+" ");  
                            left++;  
                        }  
                    }  
                }  
            }  
        }  
    }  
}
```

119. Search in Rotated Sorted Array

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {4,5,6,7,0,1,2};
        int target = 0;

        int l = 0, r = arr.length - 1;

        while (l <= r) {
            int mid = (l + r) / 2;

            if (arr[mid] == target) {
                System.out.println(mid);
                return;
            }

            if (arr[l] <= arr[mid]) {
                if (target >= arr[l] && target < arr[mid])
                    r = mid - 1;
                else l = mid + 1;
            } else {
                if (target > arr[mid] && target <= arr[r])
                    l = mid + 1;
                else r = mid - 1;
            }
        }
    }
}
```

120. Find Peak Element

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1,2,3,1};
        int l=0,r=arr.length-1;

        while(l<r){
            int mid=(l+r)/2;
            if(arr[mid]>arr[mid+1]) r=mid;
            else l=mid+1;
        }

        System.out.println(l);
    }
}
```


121. Kth Largest Element

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] arr = {3,2,1,5,6,4};
        int k = 2;

        PriorityQueue<Integer> pq = new PriorityQueue<>();

        for(int num : arr){
            pq.add(num);
            if(pq.size() > k) pq.poll();
        }

        System.out.println(pq.peek());
    }
}
```

122. Top K Frequent Elements

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] arr = {1,1,1,2,2,3};
        int k = 2;

        HashMap<Integer,Integer> map = new HashMap<>();
        for(int num : arr)
            map.put(num, map.getOrDefault(num,0)+1);

        PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)->a[1]-b[1]);

        for(int key : map.keySet()){
            pq.add(new int[]{key,map.get(key)});
            if(pq.size()>k) pq.poll();
        }

        while(!pq.isEmpty())
            System.out.print(pq.poll()[0]+" ");
    }
}
```

123. Valid Parentheses

```
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        String s = "()[]{}";
        Stack<Character> st = new Stack<>();

        for(char c : s.toCharArray()){
            if(c=='('||c=='['||c=='{')
                st.push(c);
            else{
                if(st.isEmpty()) return;
                char top = st.pop();
                if((c=='(' && top!='(') ||
                   (c=='[' && top!='[') ||
                   (c=='{' && top!='{'))
                    return;
            }
        }
        System.out.println(st.isEmpty());
    }
}
```

124. Min Stack

```
import java.util.Stack;

class MinStack {
    Stack<Integer> stack = new Stack<>();
    Stack<Integer> min = new Stack<>();

    void push(int x){
        stack.push(x);
        if(min.isEmpty() || x <= min.peek())
            min.push(x);
    }

    void pop(){
        if(stack.pop().equals(min.peek()))
            min.pop();
    }

    int getMin(){
        return min.peek();
    }
}
```

125. Evaluate Postfix Expression

```
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        String exp = "23*5+";
        Stack<Integer> st = new Stack<>();

        for(char c : exp.toCharArray()){
            if(Character.isDigit(c))
                st.push(c - '0');
            else{
                int b = st.pop();
                int a = st.pop();

                switch(c){
                    case '+': st.push(a+b); break;
                    case '*': st.push(a*b); break;
                }
            }
        }
        System.out.println(st.pop());
    }
}
```

126. Binary Tree Node Creation

```
class Node {
    int data;
    Node left, right;

    Node(int val) {
        data = val;
        left = right = null;
    }
}
```

127. Inorder Traversal (Recursive)

```
class Node {
    int data;
    Node left, right;

    Node(int val) { data = val; }
}

public class Main {
    static void inorder(Node root) {
        if (root == null) return;

        inorder(root.left);
        System.out.print(root.data + " ");
        inorder(root.right);
    }

    public static void main(String[] args) {
        Node root = new Node(1);
        root.left = new Node(2);
        root.right = new Node(3);

        inorder(root);
    }
}
```

128. Preorder Traversal

```
public class Main {
    static void preorder(Node root) {
        if (root == null) return;

        System.out.print(root.data + " ");
        preorder(root.left);
        preorder(root.right);
    }
}
```

129. Postorder Traversal

```
public class Main {
    static void postorder(Node root) {
        if (root == null) return;

        postorder(root.left);
        postorder(root.right);
        System.out.print(root.data + " ");
    }
}
```

130. Level Order Traversal (BFS)

```
import java.util.*;

public class Main {
    static void levelOrder(Node root) {
        Queue<Node> q = new LinkedList<>();
        q.add(root);

        while (!q.isEmpty()) {
            Node temp = q.poll();
            System.out.print(temp.data + " ");

            if (temp.left != null) q.add(temp.left);
            if (temp.right != null) q.add(temp.right);
        }
    }
}
```

131. Height of Binary Tree

```
public class Main {
    static int height(Node root) {
        if (root == null) return 0;
        return 1 + Math.max(height(root.left), height(root.right));
    }
}
```

132. Count Nodes in Binary Tree

```
public class Main {
    static int count(Node root) {
        if (root == null) return 0;
        return 1 + count(root.left) + count(root.right);
    }
}
```

133. Sum of Nodes

```
public class Main {
    static int sum(Node root) {
        if (root == null) return 0;
        return root.data + sum(root.left) + sum(root.right);
    }
}
```

134. Check Balanced Binary Tree

```
public class Main {  
    static boolean isBalanced(Node root) {  
        if (root == null) return true;  
  
        int lh = height(root.left);  
        int rh = height(root.right);  
  
        return Math.abs(lh - rh) <= 1 &&  
            isBalanced(root.left) &&  
            isBalanced(root.right);  
    }  
}
```

135. Diameter of Binary Tree

```
public class Main {  
    static int diameter(Node root) {  
        if (root == null) return 0;  
  
        int lh = height(root.left);  
        int rh = height(root.right);  
  
        int ld = diameter(root.left);  
        int rd = diameter(root.right);  
  
        return Math.max(lh + rh + 1, Math.max(ld, rd));  
    }  
}
```

136. Lowest Common Ancestor (Binary Tree)

```
public class Main {  
    static Node lca(Node root, int n1, int n2) {  
        if (root == null || root.data == n1 || root.data == n2)  
            return root;  
  
        Node left = lca(root.left, n1, n2);  
        Node right = lca(root.right, n1, n2);  
  
        if (left != null && right != null) return root;  
        return (left != null) ? left : right;  
    }  
}
```

137. Binary Search Tree Insert

```
public class Main {  
    static Node insert(Node root, int val) {  
        if (root == null) return new Node(val);  
  
        if (val < root.data)  
            root.left = insert(root.left, val);  
        else  
            root.right = insert(root.right, val);  
  
        return root;  
    }  
}
```

138. Search in BST

```
public class Main {  
    static boolean search(Node root, int key) {  
        if (root == null) return false;  
  
        if (root.data == key) return true;  
  
        if (key < root.data)  
            return search(root.left, key);  
        else  
            return search(root.right, key);  
    }  
}
```

139. Delete Node in BST

```
public class Main {
    static Node delete(Node root, int key) {
        if (root == null) return null;

        if (key < root.data)
            root.left = delete(root.left, key);
        else if (key > root.data)
            root.right = delete(root.right, key);
        else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;

            Node min = findMin(root.right);
            root.data = min.data;
            root.right = delete(root.right, min.data);
        }
        return root;
    }

    static Node findMin(Node root) {
        while (root.left != null)
            root = root.left;
        return root;
    }
}
```

140. Graph Representation (Adjacency List)

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int V = 5;
        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < V; i++)
            graph.add(new ArrayList<>());

        graph.get(0).add(1);
        graph.get(1).add(2);

        System.out.println(graph);
    }
}
```


141. DFS (Depth First Search)

```
import java.util.*;

public class Main {
    static void dfs(int node, boolean[] visited, ArrayList<ArrayList<Integer>> graph) {
        visited[node] = true;
        System.out.print(node + " ");

        for (int neigh : graph.get(node)) {
            if (!visited[neigh])
                dfs(neigh, visited, graph);
        }
    }
}
```

142. BFS (Graph)

```
import java.util.*;

public class Main {
    static void bfs(int start, ArrayList<ArrayList<Integer>> graph) {
        boolean[] visited = new boolean[graph.size()];
        Queue<Integer> q = new LinkedList<>();

        q.add(start);
        visited[start] = true;

        while (!q.isEmpty()) {
            int node = q.poll();
            System.out.print(node + " ");

            for (int neigh : graph.get(node)) {
                if (!visited[neigh]) {
                    visited[neigh] = true;
                    q.add(neigh);
                }
            }
        }
    }
}
```

143. Detect Cycle in Undirected Graph

```
import java.util.*;

public class Main {
    static boolean dfs(int node, int parent, boolean[] visited,
        ArrayList<ArrayList<Integer>> graph) {

        visited[node] = true;

        for (int neigh : graph.get(node)) {
            if (!visited[neigh]) {
                if (dfs(neigh, node, visited, graph))
                    return true;
            } else if (neigh != parent) {
                return true;
            }
        }
        return false;
    }
}
```

144. Number of Connected Components

```
public class Main {
    static int countComponents(int V, ArrayList<ArrayList<Integer>> graph) {
        boolean[] visited = new boolean[V];
        int count = 0;

        for (int i = 0; i < V; i++) {
            if (!visited[i]) {
                dfs(i, visited, graph);
                count++;
            }
        }
        return count;
    }
}
```

145. Flood Fill Algorithm

```
public class Main {
    static void fill(int[][] image, int sr, int sc, int newColor, int oldColor) {
        if (sr < 0 || sc < 0 || sr >= image.length || sc >= image[0].length)
            return;

        if (image[sr][sc] != oldColor) return;

        image[sr][sc] = newColor;

        fill(image, sr + 1, sc, newColor, oldColor);
        fill(image, sr - 1, sc, newColor, oldColor);
        fill(image, sr, sc + 1, newColor, oldColor);
        fill(image, sr, sc - 1, newColor, oldColor);
    }
}
```

146. Number of Islands

```
public class Main {
    static void dfs(char[][] grid, int i, int j) {
        if (i < 0 || j < 0 || i >= grid.length || j >= grid[0].length
            || grid[i][j] == '0') return;

        grid[i][j] = '0';

        dfs(grid, i+1, j);
        dfs(grid, i-1, j);
        dfs(grid, i, j+1);
        dfs(grid, i, j-1);
    }
}
```

147. Topological Sort (DFS)

```
import java.util.*;

public class Main {
    static void topo(int node, boolean[] visited,
                    Stack<Integer> st, ArrayList<ArrayList<Integer>> graph) {

        visited[node] = true;

        for (int neigh : graph.get(node)) {
            if (!visited[neigh])
                topo(neigh, visited, st, graph);
        }

        st.push(node);
    }
}
```

148. Dijkstra's Algorithm

```
import java.util.*;

public class Main {
    static void dijkstra(int src, ArrayList<ArrayList<int[]>> graph, int V) {
        int[] dist = new int[V];
        Arrays.fill(dist, Integer.MAX_VALUE);

        PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)->a[1]-b[1]);

        pq.add(new int[]{src, 0});
        dist[src] = 0;

        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            int node = cur[0], d = cur[1];

            for (int[] neigh : graph.get(node)) {
                int next = neigh[0], weight = neigh[1];

                if (d + weight < dist[next]) {
                    dist[next] = d + weight;
                    pq.add(new int[]{next, dist[next]});
                }
            }
        }
    }
}
```

149. Bellman-Ford Algorithm

```
public class Main {
    static void bellman(int V, int[][] edges, int src) {
        int[] dist = new int[V];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        for (int i = 0; i < V - 1; i++) {
            for (int[] e : edges) {
                int u = e[0], v = e[1], w = e[2];
                if (dist[u] != Integer.MAX_VALUE &&
                    dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                }
            }
        }
    }
}
```

150. Disjoint Set (Union-Find)

```
class DSU {
    int[] parent;

    DSU(int n) {
        parent = new int[n];
        for (int i = 0; i < n; i++)
            parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    void union(int a, int b) {
        parent[find(a)] = find(b);
    }
}
```

151. Fibonacci using DP (Memoization)

```
import java.util.Arrays;

public class Main {
    static int fib(int n, int[] dp) {
        if (n <= 1) return n;

        if (dp[n] != -1) return dp[n];

        return dp[n] = fib(n - 1, dp) + fib(n - 2, dp);
    }

    public static void main(String[] args) {
        int n = 6;
        int[] dp = new int[n + 1];
        Arrays.fill(dp, -1);

        System.out.println(fib(n, dp));
    }
}
```

152. Fibonacci using Tabulation

```
public class Main {
    public static void main(String[] args) {
        int n = 6;
        int[] dp = new int[n + 1];

        dp[0] = 0;
        dp[1] = 1;

        for (int i = 2; i <= n; i++)
            dp[i] = dp[i - 1] + dp[i - 2];

        System.out.println(dp[n]);
    }
}
```

153. Climbing Stairs

```
public class Main {  
    public static void main(String[] args) {  
        int n = 5;  
        int[] dp = new int[n + 1];  
  
        dp[0] = 1;  
        dp[1] = 1;  
  
        for (int i = 2; i <= n; i++)  
            dp[i] = dp[i - 1] + dp[i - 2];  
  
        System.out.println(dp[n]);  
    }  
}
```

154. House Robber Problem

```
public class Main {  
    public static void main(String[] args) {  
        int[] nums = {2,7,9,3,1};  
        int prev = 0, curr = 0;  
  
        for (int num : nums) {  
            int temp = Math.max(curr, prev + num);  
            prev = curr;  
            curr = temp;  
        }  
  
        System.out.println(curr);  
    }  
}
```

155. Coin Change (Minimum Coins)

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] coins = {1,2,5};
        int amount = 11;

        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;

        for (int coin : coins) {
            for (int i = coin; i <= amount; i++) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }

        System.out.println(dp[amount] > amount ? -1 : dp[amount]);
    }
}
```

156. Longest Increasing Subsequence

```
public class Main {
    public static void main(String[] args) {
        int[] nums = {10,9,2,5,3,7,101,18};
        int n = nums.length;
        int[] dp = new int[n];

        Arrays.fill(dp, 1);
        int max = 1;

        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[i] > nums[j])
                    dp[i] = Math.max(dp[i], dp[j] + 1);
            }
            max = Math.max(max, dp[i]);
        }

        System.out.println(max);
    }
}
```


157. Longest Common Subsequence

```
public class Main {
    public static void main(String[] args) {
        String a = "abcde", b = "ace";
        int m = a.length(), n = b.length();

        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (a.charAt(i - 1) == b.charAt(j - 1))
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        System.out.println(dp[m][n]);
    }
}
```

158. Edit Distance

```
public class Main {
    public static void main(String[] args) {
        String a = "horse", b = "ros";
        int m = a.length(), n = b.length();

        int[][] dp = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (a.charAt(i - 1) == b.charAt(j - 1))
                    dp[i][j] = dp[i - 1][j - 1];
                else
                    dp[i][j] = 1 + Math.min(dp[i - 1][j],
                                            Math.min(dp[i][j - 1], dp[i - 1][j - 1]));
            }
        }

        System.out.println(dp[m][n]);
    }
}
```

159. 0/1 Knapsack

```
public class Main {
    public static void main(String[] args) {
        int[] wt = {1,3,4,5};
        int[] val = {1,4,5,7};
        int W = 7;

        int[][] dp = new int[wt.length + 1][W + 1];

        for (int i = 1; i <= wt.length; i++) {
            for (int w = 1; w <= W; w++) {
                if (wt[i - 1] <= w)
                    dp[i][w] = Math.max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                                         dp[i - 1][w]);
                else
                    dp[i][w] = dp[i - 1][w];
            }
        }

        System.out.println(dp[wt.length][W]);
    }
}
```

160. Subset Sum Problem

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {2,3,7,8,10};
        int sum = 11;

        boolean[][] dp = new boolean[arr.length + 1][sum + 1];

        for (int i = 0; i <= arr.length; i++)
            dp[i][0] = true;

        for (int i = 1; i <= arr.length; i++) {
            for (int j = 1; j <= sum; j++) {
                if (arr[i - 1] <= j)
                    dp[i][j] = dp[i - 1][j] || dp[i - 1][j - arr[i - 1]];
                else
                    dp[i][j] = dp[i - 1][j];
            }
        }

        System.out.println(dp[arr.length][sum]);
    }
}
```

161. Partition Equal Subset Sum

```
public class Main {  
    public static void main(String[] args) {  
        int[] nums = {1,5,11,5};  
        int sum = 0;  
  
        for (int num : nums) sum += num;  
  
        if (sum % 2 != 0) {  
            System.out.println(false);  
            return;  
        }  
  
        int target = sum / 2;  
        boolean[] dp = new boolean[target + 1];  
        dp[0] = true;  
  
        for (int num : nums) {  
            for (int j = target; j >= num; j--) {  
                dp[j] = dp[j] || dp[j - num];  
            }  
        }  
  
        System.out.println(dp[target]);  
    }  
}
```

162. Minimum Path Sum (Grid)

```
public class Main {
    public static void main(String[] args) {
        int[][] grid = {{1,3,1},{1,5,1},{4,2,1}};
        int m = grid.length, n = grid[0].length;

        int[][] dp = new int[m][n];

        dp[0][0] = grid[0][0];

        for (int i = 1; i < m; i++)
            dp[i][0] = dp[i - 1][0] + grid[i][0];

        for (int j = 1; j < n; j++)
            dp[0][j] = dp[0][j - 1] + grid[0][j];

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = grid[i][j] +
                    Math.min(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        System.out.println(dp[m - 1][n - 1]);
    }
}
```

163. Unique Paths

```
public class Main {
    public static void main(String[] args) {
        int m = 3, n = 3;
        int[][] dp = new int[m][n];

        for (int i = 0; i < m; i++) dp[i][0] = 1;
        for (int j = 0; j < n; j++) dp[0][j] = 1;

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }

        System.out.println(dp[m - 1][n - 1]);
    }
}
```

164. Palindromic Substrings Count

```
public class Main {  
    public static void main(String[] args) {  
        String s = "aaa";  
        int count = 0;  
  
        for (int center = 0; center < s.length(); center++) {  
            int l = center, r = center;  
            while (l >= 0 && r < s.length() &&  
                s.charAt(l) == s.charAt(r)) {  
                count++;  
                l--; r++;  
            }  
  
            l = center; r = center + 1;  
            while (l >= 0 && r < s.length() &&  
                s.charAt(l) == s.charAt(r)) {  
                count++;  
                l--; r++;  
            }  
        }  
  
        System.out.println(count);  
    }  
}
```

165. Longest Palindromic Substring

```
public class Main {
    public static void main(String[] args) {
        String s = "babad";
        String res = "";

        for (int i = 0; i < s.length(); i++) {
            String p1 = expand(s, i, i);
            String p2 = expand(s, i, i + 1);

            if (p1.length() > res.length()) res = p1;
            if (p2.length() > res.length()) res = p2;
        }

        System.out.println(res);
    }

    static String expand(String s, int l, int r) {
        while (l >= 0 && r < s.length() &&
            s.charAt(l) == s.charAt(r)) {
            l--; r++;
        }
        return s.substring(l + 1, r);
    }
}
```

166. Decode Ways

```
public class Main {
    public static void main(String[] args) {
        String s = "226";
        int n = s.length();

        int[] dp = new int[n + 1];
        dp[0] = 1;
        dp[1] = s.charAt(0) == '0' ? 0 : 1;

        for (int i = 2; i <= n; i++) {
            int one = Integer.parseInt(s.substring(i - 1, i));
            int two = Integer.parseInt(s.substring(i - 2, i));

            if (one >= 1) dp[i] += dp[i - 1];
            if (two >= 10 && two <= 26)
                dp[i] += dp[i - 2];
        }

        System.out.println(dp[n]);
    }
}
```

167. Word Break

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        String s = "leetcode";
        Set<String> dict = new HashSet<>(Arrays.asList("leet","code"));

        boolean[] dp = new boolean[s.length() + 1];
        dp[0] = true;

        for (int i = 1; i <= s.length(); i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && dict.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }
        }

        System.out.println(dp[s.length()]);
    }
}
```

168. Longest Common Substring

```
public class Main {
    public static void main(String[] args) {
        String a = "abc", b = "bcd";
        int max = 0;

        int[][] dp = new int[a.length()+1][b.length()+1];

        for(int i=1;i<=a.length();i++){
            for(int j=1;j<=b.length();j++){
                if(a.charAt(i-1)==b.charAt(j-1)){
                    dp[i][j]=1+dp[i-1][j-1];
                    max=Math.max(max,dp[i][j]);
                }
            }
        }

        System.out.println(max);
    }
}
```

169. Rod Cutting Problem

```
public class Main {
    public static void main(String[] args) {
        int[] price = {2,5,7,8};
        int n = 4;

        int[] dp = new int[n + 1];

        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                dp[i] = Math.max(dp[i], price[j] + dp[i - j - 1]);
            }
        }

        System.out.println(dp[n]);
    }
}
```

170. Matrix Chain Multiplication

```
public class Main {
    static int solve(int[] arr, int i, int j) {
        if (i == j) return 0;

        int min = Integer.MAX_VALUE;

        for (int k = i; k < j; k++) {
            int cost = solve(arr, i, k) +
                solve(arr, k + 1, j) +
                arr[i - 1] * arr[k] * arr[j];

            min = Math.min(min, cost);
        }
        return min;
    }
}
```


171. Egg Dropping Problem

```
public class Main {
    static int eggDrop(int e, int f) {
        if (f == 0 || f == 1) return f;
        if (e == 1) return f;

        int min = Integer.MAX_VALUE;

        for (int k = 1; k <= f; k++) {
            int temp = 1 + Math.max(
                eggDrop(e - 1, k - 1),
                eggDrop(e, f - k)
            );
            min = Math.min(min, temp);
        }
        return min;
    }
}
```

172. Wildcard Matching

```
public class Main {
    public static void main(String[] args) {
        String s = "aa", p = "*";
        boolean[][] dp = new boolean[s.length()+1][p.length()+1];

        dp[0][0] = true;

        for(int j=1;j<=p.length();j++){
            if(p.charAt(j-1)=='*')
                dp[0][j]=dp[0][j-1];
        }

        for(int i=1;i<=s.length();i++){
            for(int j=1;j<=p.length();j++){
                if(p.charAt(j-1)=='*')
                    dp[i][j]=dp[i][j-1]||dp[i-1][j];
                else if(p.charAt(j-1)=='?' || s.charAt(i-1)==p.charAt(j-1))
                    dp[i][j]=dp[i-1][j-1];
            }
        }

        System.out.println(dp[s.length()][p.length()]);
    }
}
```

173. Burst Balloons

```
public class Main {
    static int solve(int[] nums, int l, int r) {
        if (l > r) return 0;

        int max = 0;

        for (int i = l; i <= r; i++) {
            int coins = nums[l - 1] * nums[i] * nums[r + 1] +
                solve(nums, l, i - 1) +
                solve(nums, i + 1, r);

            max = Math.max(max, coins);
        }
        return max;
    }
}
```

174. Palindrome Partitioning

```
public class Main {
    static int minCuts(String s, int i, int j) {
        if (i >= j || isPal(s, i, j)) return 0;

        int min = Integer.MAX_VALUE;

        for (int k = i; k < j; k++) {
            int temp = 1 + minCuts(s, i, k) + minCuts(s, k + 1, j);
            min = Math.min(min, temp);
        }
        return min;
    }

    static boolean isPal(String s, int i, int j) {
        while (i < j) {
            if (s.charAt(i++) != s.charAt(j--))
                return false;
        }
        return true;
    }
}
```

175. Boolean Parenthesization

```
public class Main {
    static int solve(String s, int i, int j, boolean isTrue) {
        if (i > j) return 0;
        if (i == j) {
            if (isTrue) return s.charAt(i) == 'T' ? 1 : 0;
            else return s.charAt(i) == 'F' ? 1 : 0;
        }

        int ways = 0;

        for (int k = i + 1; k < j; k += 2) {
            int lt = solve(s, i, k - 1, true);
            int lf = solve(s, i, k - 1, false);
            int rt = solve(s, k + 1, j, true);
            int rf = solve(s, k + 1, j, false);

            char op = s.charAt(k);

            if (op == '&') {
                if (isTrue) ways += lt * rt;
                else ways += lt * rf + lf * rt + lf * rf;
            }
        }
        return ways;
    }
}
```

176. Sliding Window Maximum

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] nums = {1,3,-1,-3,5,3,6,7};
        int k = 3;

        Deque<Integer> dq = new LinkedList<>();

        for (int i = 0; i < nums.length; i++) {
            if (!dq.isEmpty() && dq.peek() == i - k)
                dq.poll();

            while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i])
                dq.pollLast();

            dq.add(i);

            if (i >= k - 1)
                System.out.print(nums[dq.peek()] + " ");
        }
    }
}
```

177. Minimum Window Substring

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        String s = "ADOBECODEBANC", t = "ABC";
        int[] map = new int[128];

        for (char c : t.toCharArray()) map[c]++;

        int left = 0, right = 0, count = t.length();
        int minLen = Integer.MAX_VALUE, start = 0;

        while (right < s.length()) {
            if (map[s.charAt(right++)]-- > 0) count--;

            while (count == 0) {
                if (right - left < minLen) {
                    minLen = right - left;
                    start = left;
                }

                if (map[s.charAt(left++)]++ == 0) count++;
            }
        }

        System.out.println(minLen == Integer.MAX_VALUE ? "" :
            s.substring(start, start + minLen));
    }
}
```

178. Largest Rectangle in Histogram

```
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        int[] heights = {2,1,5,6,2,3};
        Stack<Integer> st = new Stack<>();
        int max = 0;

        for (int i = 0; i <= heights.length; i++) {
            int h = (i == heights.length) ? 0 : heights[i];

            while (!st.isEmpty() && h < heights[st.peek()]) {
                int height = heights[st.pop()];
                int width = st.isEmpty() ? i : i - st.peek() - 1;
                max = Math.max(max, height * width);
            }
            st.push(i);
        }

        System.out.println(max);
    }
}
```

179. Maximal Rectangle (Binary Matrix)

```
public class Main {
    public static void main(String[] args) {
        int[][] mat = {{1,0,1,0},{1,0,1,1},{1,1,1,1}};
        int[] heights = new int[mat[0].length];
        int max = 0;

        for (int[] row : mat) {
            for (int j = 0; j < row.length; j++) {
                heights[j] = (row[j] == 0) ? 0 : heights[j] + 1;
            }
            max = Math.max(max, largest(heights));
        }

        System.out.println(max);
    }

    static int largest(int[] h) {
        Stack<Integer> st = new Stack<>();
        int max = 0;

        for (int i = 0; i <= h.length; i++) {
            int height = (i == h.length) ? 0 : h[i];

            while (!st.isEmpty() && height < h[st.peek()]) {
                int ht = h[st.pop()];
                int w = st.isEmpty() ? i : i - st.peek() - 1;
                max = Math.max(max, ht * w);
            }
            st.push(i);
        }
        return max;
    }
}
```

180. Median of Two Sorted Arrays

```
public class Main {
    public static void main(String[] args) {
        int[] a = {1,3};
        int[] b = {2};

        int[] merged = new int[a.length + b.length];
        int i=0,j=0,k=0;

        while(i<a.length && j<b.length){
            if(a[i]<b[j]) merged[k++]=a[i++];
            else merged[k++]=b[j++];
        }

        while(i<a.length) merged[k++]=a[i++];
        while(j<b.length) merged[k++]=b[j++];

        int n = merged.length;

        if(n%2==0)
            System.out.println((merged[n/2-1]+merged[n/2])/2.0);
        else
            System.out.println(merged[n/2]);
    }
}
```


181. Merge K Sorted Lists

```
import java.util.*;

class ListNode {
    int val;
    ListNode next;
    ListNode(int x) { val = x; }
}

public class Main {
    static ListNode merge(ListNode[] lists) {
        PriorityQueue<ListNode> pq =
            new PriorityQueue<>((a,b)->a.val-b.val);

        for(ListNode node : lists)
            if(node!=null) pq.add(node);

        ListNode dummy = new ListNode(0), curr = dummy;

        while(!pq.isEmpty()){
            ListNode node = pq.poll();
            curr.next = node;
            curr = curr.next;

            if(node.next!=null) pq.add(node.next);
        }
        return dummy.next;
    }
}
```

182. LRU Cache

```
import java.util.*;

class LRUCache {
    int capacity;
    LinkedHashMap<Integer,Integer> map;

    LRUCache(int cap){
        capacity = cap;
        map = new LinkedHashMap<>(cap,0.75f,true){
            protected boolean removeEldestEntry(Map.Entry e){
                return size()>capacity;
            }
        };
    }

    int get(int key){
        return map.containsKey(key)?map.get(key):-1;
    }

    void put(int key,int value){
        map.put(key,value);
    }
}
```

183. LFU Cache (Basic)

```
import java.util.*;

class LFUCache {
    HashMap<Integer,Integer> map = new HashMap<>();

    void put(int key,int val){
        map.put(key,val);
    }

    int get(int key){
        return map.containsKey(key)?map.get(key):-1;
    }
}
```

184. Serialize and Deserialize Binary Tree

```
import java.util.*;

public class Main {
    static String serialize(Node root){
        if(root==null) return "null";
        return root.data + "," +
            serialize(root.left) + "," +
            serialize(root.right);
    }
}
```

185. Trie Implementation

```
class Trie {
    Trie[] children = new Trie[26];
    boolean isEnd;

    void insert(String word){
        Trie node = this;
        for(char c:word.toCharArray()){
            int i=c-'a';
            if(node.children[i]==null)
                node.children[i]=new Trie();
            node=node.children[i];
        }
        node.isEnd=true;
    }
}
```

186. Word Search (Backtracking)

```
public class Main {
    static boolean exist(char[][] board,String word,int i,int j,int k){
        if(k==word.length()) return true;
        if(i<0||j<0||i>=board.length||j>=board[0].length||
            board[i][j]!=word.charAt(k)) return false;

        char temp=board[i][j];
        board[i][j]='#';

        boolean found = exist(board,word,i+1,j,k+1) ||
            exist(board,word,i-1,j,k+1) ||
            exist(board,word,i,j+1,k+1) ||
            exist(board,word,i,j-1,k+1);

        board[i][j]=temp;
        return found;
    }
}
```

187. N-Queens Problem

```
public class Main {
    static boolean isSafe(char[][] board,int r,int c){
        for(int i=0;i<r;i++){
            if(board[i][c]=='Q') return false;

            for(int i=r-1,j=c-1;i>=0&&j>=0;i--,j--){
                if(board[i][j]=='Q') return false;

                for(int i=r-1,j=c+1;i>=0&&j<board.length;i--,j++){
                    if(board[i][j]=='Q') return false;
                }
            }
        }
    }
}
```

188. Sudoku Solver

```
public class Main {
    static boolean solve(char[][] board){
        for(int i=0;i<9;i++){
            for(int j=0;j<9;j++){
                if(board[i][j]=='.'){
                    for(char c='1';c<='9';c++){
                        if(valid(board,i,j,c)){
                            board[i][j]=c;
                            if(solve(board)) return true;
                            board[i][j]='.';
                        }
                    }
                    return false;
                }
            }
        }
        return true;
    }
}
```

189. KMP String Matching

```
public class Main {
    static void lps(String pat,int[] lps){
        int len=0,i=1;

        while(i<pat.length()){
            if(pat.charAt(i)==pat.charAt(len))
                lps[i++]=++len;
            else if(len!=0)
                len=lps[len-1];
            else lps[i++]=0;
        }
    }
}
```

190. Rabin-Karp Algorithm

```
public class Main {
    static void search(String txt,String pat){
        int m=pat.length(),n=txt.length();

        for(int i=0;i<=n-m;i++){
            if(txt.substring(i,i+m).equals(pat))
                System.out.println(i);
        }
    }
}
```

191. Z Algorithm

```
public class Main {
    static void zAlgo(String s){
        int n=s.length();
        int[] z=new int[n];

        int l=0,r=0;

        for(int i=1;i<n;i++){
            if(i<=r)
                z[i]=Math.min(r-i+1,z[i-l]);

            while(i+z[i]<n && s.charAt(z[i])==s.charAt(i+z[i]))
                z[i]++;

            if(i+z[i]-1>r){
                l=i;
                r=i+z[i]-1;
            }
        }
    }
}
```

192. Quick Sort

```
public class Main {
    static void quickSort(int[] arr,int low,int high){
        if(low<high){
            int pi=partition(arr,low,high);
            quickSort(arr,low,pi-1);
            quickSort(arr,pi+1,high);
        }
    }

    static int partition(int[] arr,int low,int high){
        int pivot=arr[high],i=low-1;

        for(int j=low;j<high;j++){
            if(arr[j]<pivot){
                i++;
                int t=arr[i];arr[i]=arr[j];arr[j]=t;
            }
        }
        int t=arr[i+1];arr[i+1]=arr[high];arr[high]=t;
        return i+1;
    }
}
```

193. Merge Sort

```
public class Main {
    static void mergeSort(int[] arr,int l,int r){
        if(l<r){
            int m=(l+r)/2;
            mergeSort(arr,l,m);
            mergeSort(arr,m+1,r);
            merge(arr,l,m,r);
        }
    }

    static void merge(int[] arr,int l,int m,int r){
        int[] temp=new int[r-l+1];
        int i=l,j=m+1,k=0;

        while(i<=m && j<=r)
            temp[k++]=arr[i]<arr[j]?arr[i++]:arr[j++];

        while(i<=m) temp[k++]=arr[i++];
        while(j<=r) temp[k++]=arr[j++];

        for(int x=0;x<temp.length;x++)
            arr[l+x]=temp[x];
    }
}
```

194. Heap Sort

```
public class Main {
    static void heapSort(int[] arr){
        int n=arr.length;

        for(int i=n/2-1;i>=0;i--){
            heapify(arr,n,i);
        }

        for(int i=n-1;i>=0;i--){
            int t=arr[0];arr[0]=arr[i];arr[i]=t;
            heapify(arr,i,0);
        }
    }

    static void heapify(int[] arr,int n,int i){
        int largest=i,l=2*i+1,r=2*i+2;

        if(l<n && arr[l]>arr[largest]) largest=l;
        if(r<n && arr[r]>arr[largest]) largest=r;

        if(largest!=i){
            int t=arr[i];arr[i]=arr[largest];arr[largest]=t;
            heapify(arr,n,largest);
        }
    }
}
```

195. Counting Sort

```
public class Main {
    static void countSort(int[] arr){
        int max=Arrays.stream(arr).max().getAsInt();
        int[] count=new int[max+1];

        for(int num:arr) count[num]++;

        int k=0;
        for(int i=0;i<count.length;i++){
            while(count[i]-- >0)
                arr[k++]=i;
        }
    }
}
```


196. Radix Sort

```
public class Main {
    static void radixSort(int[] arr){
        int max=Arrays.stream(arr).max().getAsInt();

        for(int exp=1;max/exp>0;exp*=10)
            countSort(arr,exp);
    }
}
```

197. Bucket Sort

```
import java.util.*;

public class Main {
    static void bucketSort(float[] arr){
        int n=arr.length;

        List<Float>[] buckets=new ArrayList[n];

        for(int i=0;i<n;i++)
            buckets[i]=new ArrayList<>();

        for(float num:arr)
            buckets[(int)(num*n)].add(num);

        for(List<Float> b:buckets)
            Collections.sort(b);

        int k=0;
        for(List<Float> b:buckets)
            for(float num:b)
                arr[k++]=num;
    }
}
```

198. Bit Manipulation (Count Set Bits)

```
public class Main {
    public static void main(String[] args) {
        int n = 5, count = 0;

        while (n > 0) {
            count += n & 1;
            n >>= 1;
        }

        System.out.println(count);
    }
}
```

199. Power of Two Check

```
public class Main {  
    public static void main(String[] args) {  
        int n = 8;  
        System.out.println((n & (n - 1)) == 0);  
    }  
}
```

200. Single Number (XOR)

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {2,2,1};  
  
        int res = 0;  
        for (int num : arr)  
            res ^= num;  
  
        System.out.println(res);  
    }  
}
```